

Technical Architecture

Technical Architecture & MVC Design Documentation

Technical Architecture Overview

The Book a Doctor App features a modern technical architecture based on a client-server model. The frontend utilises Bootstrap and Material UI for a responsive user interface, with Axios handling seamless API communication. The backend is powered by Express.js, offering robust server-side logic, while MongoDB provides scalable data storage for user profiles, appointments, and doctor information. Authentication is secured using JWT for session management and bcrypt for password hashing. Moment.js manages date and time functionalities, ensuring accurate appointment scheduling. The admin interfaces overseas doctor registration, platform governance, and ensures compliance, with Role-based Access Control (RBAC) managing access levels. Scalability is supported by MongoDB, and performance optimization is achieved with load balancing and caching techniques.

The Book a doctor backend application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability for this application.

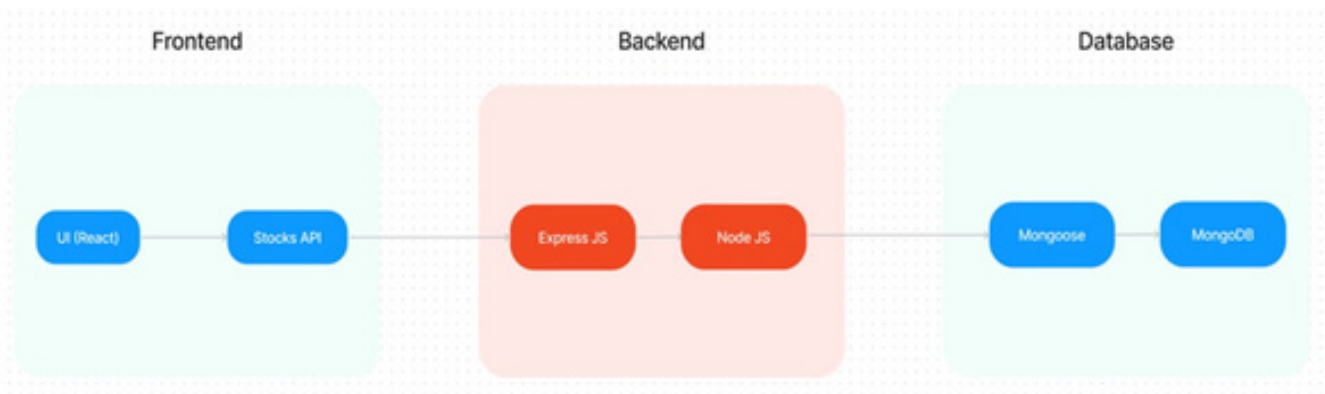


Figure 1: Application Technical Architecture and Component Flow

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.

Scalability: New features can be added easily by creating new routes, controllers, and models.

Reusability: Logic in controllers and models can be reused across multiple parts of the application.

Testing: Each layer can be tested independently, especially the controllers and models.

Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.